

NotchPay

Guide d'intégration de l'API de paiement **Node.js & Python**

v2025

REST API

1. Introduction à NotchPay

NotchPay est une plateforme de paiement africaine qui permet aux entreprises d'accepter des paiements via Mobile Money (MTN, Orange, etc.), cartes bancaires et d'autres méthodes locales. L'API REST de NotchPay est construite autour des principes HTTP standards et retourne des réponses au format JSON.

1.1 Informations essentielles

URL de base	<code>https://api.notchpay.co</code>
Format de réponse	JSON
Authentification	Header <code>Authorization: YOUR_PUBLIC_KEY</code>
Clé Sandbox	Commence par <code>sb.</code>
Clé Production	Commence par <code>b.</code>

Obtenez vos clés API depuis le tableau de bord :
<https://business.notchpay.co/developer/api-keys>

1.2 Codes de statut HTTP

Code	Statut	Description
200	OK	Requête traitée avec succès
201	Created	Ressource créée avec succès
202	Accepted	Requête acceptée pour traitement
400	Bad Request	Paramètres invalides ou manquants
401	Unauthorized	Clé API manquante ou invalide
404	Not Found	Ressource introuvable
422	Unprocessable Entity	Erreur sémantique dans la requête
429	Too Many Requests	Limite de taux dépassée
500	Server Error	Erreur côté serveur NotchPay

2. Intégration Node.js

2.1 Installation et configuration

Installez les dépendances nécessaires dans votre projet Node.js :

```
bash
npm install axios dotenv
# ou avec yarn
yarn add axios dotenv
```

Créez un fichier `.env` à la racine de votre projet :

```
.env
NOTCHPAY_PUBLIC_KEY=sb.votre_cle_publique_sandbox
NOTCHPAY_PRIVATE_KEY=votre_cle_privee
NOTCHPAY_BASE_URL=https://api.notchpay.co
```

Ne commitez jamais votre fichier `.env` dans votre dépôt Git. Ajoutez-le au `.gitignore`.

2.2 Client HTTP de base

Créez un fichier `notchpay.js` pour centraliser les appels à l'API :

```
javascript
// notchpay.js
const axios = require('axios');
require('dotenv').config();

const notchpayClient = axios.create({
  baseURL: process.env.NOTCHPAY_BASE_URL || 'https://api.notchpay.co',
  headers: {
    'Authorization': process.env.NOTCHPAY_PUBLIC_KEY,
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
});

module.exports = notchpayClient;
```

2.3 Initialiser un paiement

L'initialisation d'un paiement retourne une `authorization_url` vers laquelle vous devez rediriger votre client :

```
javascript
// payments.js
const notchpayClient = require('./notchpay');
```

```
async function initializePayment(paymentData) {
  try {
    const response = await notchpayClient.post('/payments', {
      amount: paymentData.amount,          // Montant en centimes (ex: 5000 =
5000 XAF)
      currency: paymentData.currency,      // 'XAF', 'XOF', 'USD', etc.
      email: paymentData.email,           // Email du client
      phone: paymentData.phone,           // Téléphone du client (optionnel)
      description: paymentData.description,
      reference: paymentData.reference || `order_${Date.now()}`,
      callback: paymentData.callback,      // URL de redirection après paiement
    });

    const { transaction, authorization_url } = response.data;

    console.log('Paieement initialisé:', transaction.reference);
    console.log('Rediriger vers:', authorization_url);

    return {
      success: true,
      transaction,
      authorization_url,
    };
  } catch (error) {
    const errData = error.response?.data;
    console.error('Erreur initialisation paiement:', errData || error.message);
    return { success: false, error: errData || error.message };
  }
}

// Utilisation
initializePayment({
  amount: 5000,
  currency: 'XAF',
  email: 'client@exemple.com',
  phone: '+237670000000',
  description: 'Commande #001',
  reference: 'order_001',
  callback: 'https://monsite.com/callback',
});
```

2.4 Vérifier un paiement

Toujours vérifier le statut du paiement via l'API avant d'honorer une commande :

```
javascript
async function verifyPayment(reference) {
  try {
    const response = await notchpayClient.get(`/payments/${reference}`);
    const { transaction } = response.data;

    console.log('Statut du paiement:', transaction.status);

    // Statuts possibles: pending | processing | complete | failed | canceled |
expired
    switch (transaction.status) {
      case 'complete':
```

```
        return { success: true, transaction, message: 'Païement réussi' };
      case 'processing':
        return { success: false, transaction, message: 'Païement en cours' };
      case 'failed':
        return { success: false, transaction, message: 'Païement échoué' };
      case 'canceled':
        return { success: false, transaction, message: 'Païement annulé' };
      default:
        return { success: false, transaction, message: 'Païement en attente' };
    }
  } catch (error) {
    console.error('Erreur vérification:', error.response?.data ||
error.message);
    return { success: false, error: error.response?.data };
  }
}
```

2.5 Traiter un paiement Mobile Money

Pour initier un débit Mobile Money directement (sans redirection), utilisez l'endpoint de traitement :

```
javascript
async function processPayment(reference, channel, phone) {
  try {
    // Canaux disponibles: 'cm.mtn', 'cm.orange', 'sn.orange', 'ci.mtn', etc.
    const response = await notchpayClient.post(`/payments/${reference}`, {
      channel: channel,          // ex: 'cm.mtn' pour MTN Cameroun
      data: {
        phone: phone,           // ex: '+237680000000'
      }
    });
  } catch (error) {
    console.error('Erreur traitement:', error.response?.data);
    return { success: false, error: error.response?.data };
  }
}

// Exemple: MTN Mobile Money Cameroun
processPayment('order_001', 'cm.mtn', '+237680000000');
```

2.6 Annuler un paiement

```
javascript
async function cancelPayment(reference) {
  try {
    const response = await notchpayClient.delete(`/payments/${reference}`);
    console.log('Païement annulé:', response.data.message);
    return { success: true };
  } catch (error) {
    console.error('Erreur annulation:', error.response?.data);
    return { success: false, error: error.response?.data };
  }
}
```

```
}

// Note: Seuls les paiements en statut 'pending' peuvent être annulés
```

2.7 Lister les paiements

```
javascript
async function listPayments({ page = 1, limit = 30, status, dateStart, dateEnd
} = {}) {
  try {
    const params = { page, limit };
    if (status) params.status = status;
    if (dateStart) params.date_start = dateStart; // Format: 'YYYY-MM-DD'
    if (dateEnd) params.date_end = dateEnd;

    const response = await notchpayClient.get('/payments', { params });
    const { items, totals, current_page, last_page } = response.data;

    console.log(`${totals} paiements trouvés (page
    ${current_page}/${last_page})`);
    return { success: true, items, totals, current_page, last_page };
  } catch (error) {
    console.error('Erreur liste:', error.response?.data);
    return { success: false, error: error.response?.data };
  }
}

// Exemples d'utilisation
listPayments({ status: 'complete', dateStart: '2025-01-01' });
```

2.8 Gestion des Webhooks (Express.js)

Les webhooks permettent de recevoir des notifications en temps réel sur l'état des paiements. Implémentez un endpoint dans votre application Express :

```
javascript
// webhook.js - avec Express.js
const express = require('express');
const crypto = require('crypto');
const router = express.Router();

// Middleware pour récupérer le body brut (nécessaire pour la signature)
router.use(express.raw({ type: 'application/json' }));

router.post('/webhook/notchpay', (req, res) => {
  const signature = req.headers['x-notch-signature'];
  const payload = req.body;

  // Vérification de la signature (recommandé en production)
  // const expectedSig = crypto
  //   .createHmac('sha256', process.env.NOTCHPAY_WEBHOOK_SECRET)
  //   .update(payload).digest('hex');
  // if (signature !== expectedSig) return res.status(401).send('Signature
  invalide');
```

```
const event = JSON.parse(payload.toString());

switch (event.type) {
  case 'payment.complete':
    console.log('Païement réussi:', event.data.reference);
    // Honorer la commande ici
    break;

  case 'payment.failed':
    console.log('Païement échoué:', event.data.reference);
    // Notifier le client, libérer le stock réservé
    break;

  case 'payment.canceled':
    console.log('Païement annulé:', event.data.reference);
    break;

  default:
    console.log('Événement non géré:', event.type);
}

// Toujours répondre 200 rapidement
res.status(200).send('OK');
});

module.exports = router;
```

2.9 Gestion du callback

Lorsque le client termine le paiement, NotchPay le redirige vers votre callback URL avec la référence en paramètre :

```
javascript
// Exemple avec Express.js
app.get('/callback', async (req, res) => {
  const { reference } = req.query;

  if (!reference) {
    return res.status(400).send('Référence manquante');
  }

  const result = await verifyPayment(reference);

  if (result.success) {
    // Paiement confirmé - honorer la commande
    await fulfillOrder(reference);
    return res.redirect('/success?ref=' + reference);
  } else {
    // Paiement non complété
    return res.redirect('/failed?ref=' + reference);
  }
});
```

3. Intégration Python

3.1 Installation et configuration

```
bash
pip install requests python-dotenv
# ou avec pip3
pip3 install requests python-dotenv
```

Créez un fichier `.env` à la racine de votre projet :

```
.env
NOTCHPAY_PUBLIC_KEY=sb.votre_cle_publique_sandbox
NOTCHPAY_PRIVATE_KEY=votre_cle_privee
NOTCHPAY_BASE_URL=https://api.notchpay.co
```

3.2 Client HTTP de base

Créez un fichier `notchpay.py` pour centraliser les appels à l'API :

```
python
# notchpay.py
import os
import requests
from dotenv import load_dotenv

load_dotenv()

class NotchPayClient:
    def __init__(self):
        self.base_url = os.getenv('NOTCHPAY_BASE_URL',
        'https://api.notchpay.co')
        self.public_key = os.getenv('NOTCHPAY_PUBLIC_KEY')
        self.private_key = os.getenv('NOTCHPAY_PRIVATE_KEY')
        self.session = requests.Session()
        self.session.headers.update({
            'Authorization': self.public_key,
            'Content-Type': 'application/json',
            'Accept': 'application/json',
        })

    def get(self, endpoint, params=None):
        url = f'{self.base_url}{endpoint}'
        response = self.session.get(url, params=params)
        response.raise_for_status()
        return response.json()

    def post(self, endpoint, data=None):
        url = f'{self.base_url}{endpoint}'
        response = self.session.post(url, json=data)
        response.raise_for_status()
        return response.json()

    def delete(self, endpoint):
```

```
url = f'{self.base_url}{endpoint}'
response = self.session.delete(url)
response.raise_for_status()
return response.json()

# Instance globale
client = NotchPayClient()
```

3.3 Initialiser un paiement

```
python
# payments.py
import time
from notchpay import client

def initialize_payment(amount, currency, email, description,
                      phone=None, reference=None, callback=None):
    """
    Initialise un paiement NotchPay.
    Retourne l'autorization_url pour rediriger le client.
    """
    payload = {
        'amount': amount,
        'currency': currency,
        'email': email,
        'description': description,
        'reference': reference or f'order_{int(time.time())}',
    }
    if phone:
        payload['phone'] = phone
    if callback:
        payload['callback'] = callback

    try:
        data = client.post('/payments', payload)
        transaction = data['transaction']
        authorization_url = data['authorization_url']

        print(f" Paiement initialisé: {transaction['reference']}")
        print(f" URL de paiement: {authorization_url}")

        return {
            'success': True,
            'transaction': transaction,
            'authorization_url': authorization_url,
        }
    except Exception as e:
        print(f'Erreur initialisation: {e}')
        return {'success': False, 'error': str(e)}

# Utilisation
if __name__ == '__main__':
    result = initialize_payment(
        amount=5000,
        currency='XAF',
        email='client@exemple.com',
        phone='+237670000000',
        description='Commande #001',
        reference='order_001',
        callback='https://monsite.com/callback',
    )
```

```
print(result)
```

3.4 Vérifier un paiement

```
python
def verify_payment(reference):
    """Vérifie le statut d'un paiement par sa référence."""
    try:
        data = client.get(f'/payments/{reference}')
        transaction = data['transaction']
        status = transaction['status']

        messages = {
            'complete': 'Paielement réussi',
            'processing': 'Paielement en cours de traitement',
            'failed': 'Paielement échoué',
            'canceled': 'Paielement annulé',
            'expired': 'Paielement expiré',
            'pending': 'Paielement en attente',
        }

        return {
            'success': status == 'complete',
            'status': status,
            'message': messages.get(status, 'Statut inconnu'),
            'transaction': transaction,
        }
    except Exception as e:
        return {'success': False, 'error': str(e)}
```

3.5 Traiter un paiement Mobile Money

```
python
def process_payment(reference, channel, phone):
    """
    Lance le débit Mobile Money directement.
    Canaux: 'cm.mtn', 'cm.orange', 'sn.orange', 'ci.mtn'...
    """
    try:
        data = client.post(f'/payments/{reference}', {
            'channel': channel,
            'data': {'phone': phone},
        })
        return {'success': True, 'transaction': data['transaction']}
    except Exception as e:
        print(f'Erreur traitement paiement: {e}')
        return {'success': False, 'error': str(e)}

# Exemples
process_payment('order_001', 'cm.mtn', '+237680000000') # MTN Cameroun
process_payment('order_002', 'cm.orange', '+237699000000') # Orange Cameroun
```

3.6 Annuler et lister les paiements

```
python
def cancel_payment(reference):
    """Annule un paiement en statut 'pending'."""
    try:
        data = client.delete(f'/payments/{reference}')
        return {'success': True, 'message': data.get('message')}
    except Exception as e:
        return {'success': False, 'error': str(e)}

def list_payments(page=1, limit=30, status=None, date_start=None,
date_end=None):
    """Récupère la liste des paiements avec filtres optionnels."""
    params = {'page': page, 'limit': limit}
    if status:      params['status'] = status
    if date_start:  params['date_start'] = date_start  # Format YYYY-MM-DD
    if date_end:    params['date_end'] = date_end

    try:
        data = client.get('/payments', params=params)
        return {
            'success': True,
            'items':      data['items'],
            'total':      data['totals'],
            'current_page': data['current_page'],
            'last_page':   data['last_page'],
        }
    except Exception as e:
        return {'success': False, 'error': str(e)}

# Exemples
cancel_payment('order_001')
list_payments(status='complete', date_start='2025-01-01')
```

3.7 Gestion des Webhooks (Flask)

Voici comment recevoir et valider les webhooks NotchPay avec Flask :

```
python
# webhook_handler.py - avec Flask
import hmac
import hashlib
import json
import os
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/webhook/notchpay', methods=['POST'])
def notchpay_webhook():
    payload = request.get_data()
    signature = request.headers.get('X-Notch-Signature', '')

    # Vérification de la signature (recommandé en production)
    # secret = os.getenv('NOTCHPAY_WEBHOOK_SECRET', '').encode()
    # expected = hmac.new(secret, payload, hashlib.sha256).hexdigest()
```

```
# if not hmac.compare_digest(signature, expected):
#     return jsonify({'error': 'Signature invalide'}), 401

event = json.loads(payload)
event_type = event.get('type')
data = event.get('data', {})

if event_type == 'payment.complete':
    reference = data.get('reference')
    print(f'Païement réussi: {reference}')
    # Déclencher la livraison / accès produit
    handle_payment_complete(reference)

elif event_type == 'payment.failed':
    reference = data.get('reference')
    print(f'Païement échoué: {reference}')
    handle_payment_failed(reference)

elif event_type == 'payment.canceled':
    reference = data.get('reference')
    print(f'Païement annulé: {reference}')

# Toujours retourner 200 rapidement
return jsonify({'status': 'ok'}), 200

def handle_payment_complete(reference):
    # Votre logique métier ici
    print(f'Livraison déclenchée pour: {reference}')

def handle_payment_failed(reference):
    print(f'Notification d\'échec pour: {reference}')
```

3.8 Gestion du callback (Django)

```
python
# views.py - avec Django
from django.http import HttpResponseRedirect, HttpResponseBadRequest
from django.views import View
from .payments import verify_payment

class PaymentCallbackView(View):
    def get(self, request):
        reference = request.GET.get('reference')

        if not reference:
            return HttpResponseBadRequest('Référence manquante')

        result = verify_payment(reference)

        if result['success']:
            # Honorer la commande
            # fulfill_order(reference)
            return HttpResponseRedirect(f'/success/?ref={reference}')
        else:
            return HttpResponseRedirect(f'/failed/?ref={reference}')

# urls.py
```

```
from django.urls import path
from .views import PaymentCallbackView

urlpatterns = [
    path('callback/', PaymentCallbackView.as_view(), name='payment-callback'),
]
```

4. Bonnes pratiques & conseils

4.1 Statuts des paiements

Statut	Description	Action recommandée
pending	Initialisé, en attente	<i>Attendre ou afficher la page de paiement</i>
processing	Débit Mobile Money en cours	<i>Attendre la confirmation (polling ou webhook)</i>
complete	Paieement confirmé	<i>Honorer la commande / livrer</i>
failed	Paieement refusé ou échoué	<i>Notifier le client, proposer une nouvelle tentative</i>
canceled	Annulé par le marchand	<i>Libérer les ressources réservées</i>
expired	Délai de paiement dépassé	<i>Demander au client de recommencer</i>

4.2 Recommandations de sécurité

- Ne jamais exposer votre clé API côté client (frontend). Faites toujours les appels depuis votre serveur.
- Vérifiez systématiquement le statut du paiement via l'API avant d'honorer une commande, même si le callback ou le webhook indique un succès.
- Utilisez des références de paiement uniques et idempotentes pour éviter les doublons.
- Implémentez la vérification de signature des webhooks en production.
- Stockez les références de transactions dans votre base de données pour la réconciliation.
- Utilisez l'environnement Sandbox (clés sb.) pour tous vos tests avant la mise en production.

4.3 Gestion du Rate Limiting

L'API NotchPay limite le nombre de requêtes. En cas de dépassement (code 429), un header `Retry-After` indique le délai d'attente. Implémentez un backoff exponentiel :

```
javascript
// Node.js - Retry avec backoff exponentiel
async function apiCallWithRetry(fn, maxRetries = 3) {
  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await fn();
    } catch (error) {
      if (error.response?.status === 429 && attempt < maxRetries - 1) {
```

```
const retryAfter = error.response.headers['retry-after'] || (2 **
attempt);
console.log(`Rate limited. Attente de ${retryAfter}s...`);
await new Promise(r => setTimeout(r, retryAfter * 1000));
} else {
  throw error;
}
}
```

4.4 Canaux de paiement Mobile Money

Canal	Opérateur	Pays
cm.mtn	MTN Mobile Money	Cameroun
cm.orange	Orange Money	Cameroun
sn.orange	Orange Money	Sénégal
ci.mtn	MTN Mobile Money	Côte d'Ivoire
ci.orange	Orange Money	Côte d'Ivoire
bf.orange	Orange Money	Burkina Faso

Consultez la documentation officielle pour la liste complète des canaux disponibles :
<https://developer.notchpay.co/api-reference/resources>

5. Ressources utiles

Documentation officielle	https://developer.notchpay.co
Référence API	https://developer.notchpay.co/api-reference
Tableau de bord	https://business.notchpay.co
SDK JS officiel	https://github.com/notchpay/notchpay-js
Support email	hello@notchpay.co

Ce guide couvre les fonctionnalités principales de l'API NotchPay. Pour les transferts d'argent (Send Money), la gestion des clients, des bénéficiaires et la consultation du solde, consultez la documentation officielle à l'adresse developer.notchpay.co/api-reference.

Guide rédigé en avril 2025 • NotchPay REST API